

RESEARCH BY SID MACHINES

Technical Supplement and Reproduction Guide

Technical supplement

Research directed by the company owner

PRIVATE RESEARCH

This report is privately circulated by the owner of SID Machines for authorized recipients only. It is not intended for public distribution. Do not reproduce, publish, or forward it without the owner's permission.

Technical Supplement and Reproduction Guide

Companion to Memory Selection and Planning Budgets in Learning Agents

1 Reproduce the experiments

Use Python 3.12 or later. Install the experiment requirements, then run the commands below from the 02_Experiments directory. The supplied results are already complete; rerunning study.py overwrites runs.csv and its metadata. Copy the package before experimenting if you want to preserve the original run files.

```
python -m pip install -r requirements.txt
python -m unittest -v test_study
python study.py --workers 1
python analyze.py
python neural_transfer.py
python analyze_neural.py
python verify_results.py
```

Training requires only NumPy. Analysis and charts require Matplotlib. The supplied agent results use Python 3.12.14 and NumPy 2.3.5; figures and bootstrap analysis were produced with Python 3.13, NumPy 2.3.3, and Matplotlib 3.10.6. Actual versions are saved in the metadata. A single consistent environment satisfying requirements.txt can run the full pipeline; tiny floating-point or runtime differences across systems are possible.

The PDF and editable HTML sources are generated by build_documents.py using the saved findings. To rebuild them, install requirements_documents.txt and run python build_documents.py. PDFs were rendered and visually checked after generation. HTML versions contain the same report text and use relative image paths.

2 File map

Start with 00_START_HERE.html. The numbered folders contain the paper, experiment code, results, figures, research notes, and unchanged original project, respectively. MANIFEST_SHA256.txt records the packaged file hashes. The contents page links directly to the main documents, data, and reproduction entry points.

3 Counts and independence

The main study contains 5 environments x 20 seed values x 15 configurations = 1,500 runs. The planning ablation adds 5 x 20 x 3 retention rules x 2 planning settings = 600 runs. There are 6.3 million real training interactions and 21,000 evaluation checkpoints. The neural extension adds 20 x 7 = 140 model-training runs, 224,000 arriving examples, and 560 checkpoints. The complete package therefore contains 2,240 completed training runs.

Seed values are paired across methods, and some environments deliberately share structure. The runs are separately initialized repetitions, not 2,240 independent problem definitions. Bootstrap analysis treats seed indices as paired blocks. No confidence interval pools the neural classifier and control-agent scores.

4 Algorithms and evaluation details

Transition model

Sparse model keys contain (state, action, next state, terminal). Each key stores an observed count and reward sum. Dividing key counts by the total count for the state-action pair estimates the outcome distribution. Dividing reward sums by the same denominator estimates the contribution to expected immediate reward. Removing a buffer item subtracts exactly its contribution, and empty entries are deleted.

Coverage buffer pseudocode

```
append new transition
increment count for its state-action group
if length exceeds capacity:
  find maximum group occupancy
  scan records from oldest to newest
  remove first record in a maximum-size group
  decrement its group and model counts
```

The reservoir uses Algorithm R: the t-th arrival is inserted into a full M-item buffer with probability M/t, replacing a uniform slot. Unlike recent memory, old items can persist indefinitely. Each retained item has equal inclusion probability in the observed stream, but the stream itself is policy-dependent and may emphasize unhelpful parts of the environment.

Exact finite-horizon evaluator

For a fixed greedy policy, define success probability at horizon zero as zero. At each subsequent horizon, sum the probability of immediate terminal success and the probability of each nonterminal outcome multiplied by its previous-horizon success probability. Expected undiscounted return is computed analogously, adding immediate reward before continuation. Repeat for 100 steps and read the start-state values. Only the evaluator accesses the true transition kernel. It never modifies the agent.

This evaluator is checked against a closed-form three-step geometric-success example. Additional tests enforce buffer capacity, reconstruct model counts and rewards from retained records after every insertion, check terminal bootstrapping, verify seed repeatability, and ensure all test environments have a near-certain reachable successful policy.

Primary statistic

For each run, trapezoidal integration over the ten registered checkpoints is divided by 3,000. An area of 1 would mean perfect 100-step success throughout training. The primary difference is coverage area minus recent-memory area at capacity 128. Differences are averaged over four stationary tasks within each seed block, and 20 blocks are resampled with replacement 10,000 times. Percentiles 2.5 and 97.5 form the interval. This is uncertainty over training randomness, conditional on the fixed suite.

Endpoint caution

Success within 100 steps is forgiving once a policy can reach the goal. Expected return is retained as a secondary efficiency measure because unnecessary steps reduce it. The primary success-area statistic captures learning speed but does not fully describe path quality. A 100% success ceiling should not be interpreted as identical behavior.

5 Data dictionary and accounting

File and fields	Meaning
runs.csv identifiers	environment, method, capacity, seed, sweeps, every, family identify one training run. Capacity 0 means no raw buffer for Q learning or full model.
step and phase	Completed real training interactions and environment phase. Step 1500 is still the old goal; step 1750 is the first measured new-goal checkpoint.
success and return_value	Exact 100-step greedy success probability and expected undiscounted cumulative reward.
train_seconds	Cumulative training time excluding evaluations and deep memory inspection.
backups and plans	Cumulative scalar action-value updates and planning events. Backups are not hardware operations.
agent_bytes	Raw deep Python object-size estimate; see the array-base correction below.
retained and retained_pairs	Stored raw records and distinct retained state-action groups.
model_edges and model_pairs	Distinct model outcomes and state-action pairs represented in sparse counts.
run_metrics.csv	Per-run areas, final performance, corrected state bytes, work counts, and timing.
summary_by_environment.csv	Means and 95% bootstrap intervals for all 105 environment/configuration combinations.
paired_effects.csv	Exploratory within-environment coverage-minus-baseline contrasts.
neural_runs.csv	Per-block test scores, class-specific accuracy, examples processed, and replay-buffer bytes.

The raw recursive counter visits ndarray objects but does not follow their base attribute. Model planning creates exactly one two-dimensional action-value view over a one-dimensional float64 allocation. On the execution runtime, analysis adds 112 + 8 x states x actions bytes after the first plan. Q-learning tables own their data and need no correction. Both the correction and final corrected estimate are explicit columns; raw source data are unchanged.

Memory estimates include retained Python bookkeeping but not temporary allocations or full process RSS. The neural memory_bytes field measures the replay buffer only, not network parameters or transient gradients; it is intentionally not pooled with the control-agent state estimate. Timing confidence intervals describe variation among the observed runs and cannot compensate for hardware or implementation bias.

6 Changes from the supplied project

The expansion replaces episode budgets with exact interaction budgets, uses consistent planning settings within comparisons, records terminal outcomes explicitly, and evaluates policies exactly. It adds stronger replay and retention baselines, multiple environments, a changed objective, planning-budget ablations, uncertainty estimates, total-state accounting, and a separate neural experiment. The original project is preserved rather than silently rewritten. Its old figures are historical artifacts and are not used as evidence for the new paper.

7 Proposed local LLM validation

This section is an unexecuted protocol for a follow-up study. No downloaded language model, model API, or LLM-generated action was used in the reported experiments. The finite-state tool workflow tests action prerequisites and retries only; it does not test language interpretation or hallucination.

Task and model controls

Use two frozen locally hosted instruction models that fit the available hardware. Record checkpoint hashes, quantization, context limit, decoding parameters, and inference runtime. Build 30 held-out tool tasks in three families: structured-data transformation, file operations in an isolated workspace, and multi-step query workflows. Use a separate calibration set to fix prompts and memory rules. Include changed tool versions in a designated adaptation subset.

Compare four conditions with identical models, action prompts, and tool contracts: no cross-task memory, recent memory, reservoir memory, and coverage-based memory. Within-episode action history must be handled identically in all conditions. Set the stored and retrieved memory limits in tokens, not record counts. Coverage groups should use only observable task features; their construction cost must be counted. Do not introduce an extra LLM summarizer to one condition without charging its cost.

Data and leakage controls

Reset cross-task memory at the beginning of each independently seeded task sequence. All methods see the same task ordering. Store only feedback available to the acting agent; hidden evaluator solutions and held-out success labels must never become memory inputs. A reused task template must not reveal the exact held-out answer. Version-tag each tool outcome so obsolete records can be recognized. Record all failed calls, retries, parsing failures, and timeouts.

Metrics and prospective decision rule

For each model, compare success with a paired task bootstrap and report token usage, tool calls, invalid actions, latency, memory size, and retrieval cost. The proposed efficiency criterion is a lower 95% confidence bound on success difference above -3 percentage points, together with at least 20% lower measured total inference tokens than a recent-memory baseline. These are design targets for a new study, not achieved results. A 30-task test may be underpowered; report an inconclusive outcome rather than treating non-significance as equivalence.

Start with a free local feasibility pilot to measure runtime and task difficulty. The final experiment would require new held-out tasks after any pilot-driven tuning. If the hardware cannot execute a suitable model, keep this application as proposed work instead of substituting scripted behavior and calling it an LLM.

8 Research claims that remain open

Open questions include retention under noisy state abstraction, partial observability, adaptive memory budgets, hybrid recent and diverse archives, and stronger compute-matched baselines. Evidence from this package does not establish parameter compression, lower pretraining GPU cost, energy savings, safety of autonomous external actions, production reliability, or superiority to current foundation models. Those claims require different measurements and experiments.